

Security Audit Report

dYdX 2026 Q4: Canonical USDC

Authors:

Manuel Bravo

Oliver Bundalo

Aleksandar Ljahovic

Igor Crevar

Nikola Jovicevic

Last Revised:

12.11.2026

Contents

Security Audit Report 1
Contents 2
Audit Overview 3
Scope of this report 3
Audit plan 3
Conclusions 3
Audit Dashboard 4
Target Summary 4
Engagement Summary 4
Severity Summary 4
System Overview 5
Modes 5
Ledger 5
Deposits 5
Withdrawals 5
Settlement 6
Limits and queries 6
Threat Inspection 7
Properties 7
Findings 16
Noble Withdrawal from an Uppercase Address Never Settles 17
Pending Cap Exhaustion 19
Withdraw to Injective and Redeposit Loop Drains the Shared uusdc Rate-Limit Capacity 21
Memo Calls on Injective Run Under One Identity Shared by All Canonical Withdrawers 23
Failed Injective Withdrawals Allow injective_backing to Exceed migration_ceiling Without Limit 25
Events Are Incomplete for Indexers 27
Code Improvements 29
Appendix: Vulnerability Classification 30
Impact Score 30
Exploitability Score 31
Severity Score 32
Disclaimer 33

Audit Overview

In November 2026, dYdX engaged Informal Systems to continue the partnership and conduct a security audit of the Canonical USDC `x/canonicalusdc` module.

Scope of this report

The audit focused on evaluating the correctness and security properties of the `x/canonicalusdc` module, which allows Injective-origin USDC to back the existing trading USDC denom alongside Noble USDC. It also assessed the module's interaction with the IBC transfer stack and `x/ratelimit`. The review covered deposit, withdrawal, and settlement flows, as well as pending settlement queries.

A key priority was ensuring that trading USDC remains fully backed, that Injective and Noble transfers cannot bypass the module, and that every in-flight transfer can be settled.

Audit plan

The audit was conducted between November 2nd, 2026 and November 9th, 2026 by the following personnel:

- Manuel Bravo: Project Delivery Manager
- Aleksandar Ljahovic: Security Engineer
- Oliver Bundalo: Security Engineer
- Igor Crevar: Security Engineer
- Nikola Jovicevic: Security Engineer

Conclusions

The module design is clear and well structured. A single ledger tracks Noble backing, Injective backing, and in-flight transfers. All routed transfers go through a single transfer decorator, while an ICS4 gate prevents trading USDC and module-owned Injective USDC from leaving dYdX without passing through the module. The audit team found that the ledger remains consistent with the token supply and module balances across deposit, withdrawal, and settlement flows.

The most significant issues concern withdrawal availability. In particular, the shared pending settlement limit and the trading USDC rate-limit capacity can both be exhausted, preventing further withdrawals. Other findings relate to how user memos are handled on the Injective route, enforcement of the migration ceiling, event completeness, and code quality. All findings are detailed in the Findings table.

Audit Dashboard

Target Summary

- **Type**: Protocol and Implementation
- **Platform**: Go
- **Artifacts:** `x/canonicalusdc` module over 71edbea commit and changes in other modules visible in the PRs.

Engagement Summary

- **Dates:** November 2, 2026 - November 9, 2026
- **Method**: code review

Severity Summary

Finding Severity	#
Critical	1
High	1
Medium	2
Low	1
Informational	2
Total	7

System Overview

Trading USDC on dYdX (`uusdc`) is currently an IBC voucher of USDC bridged from Noble. The `x/canonicalusdc` module allows Injective-origin USDC to back the same trading USDC denom. This enables users to deposit and withdraw through Injective while existing balances, markets, and integrations continue to use the unchanged denom. During the migration, trading USDC is backed by both Noble USDC and Injective USDC held by the module.

Modes

The module operates in one of three modes, controlled by governance:

- **DISABLED:** The existing transfer stack behaves exactly as before.
- **GRADUAL:** Injective deposits and withdrawals are routed through the module, while Noble deposits are rejected.
- **PAUSED:** New canonical transfers are rejected, but transfers already in flight can still be settled.

Ledger

The module maintains a ledger with four amounts:

- **Noble backing:** Trading USDC backed by Noble USDC.
- **Injective backing:** Trading USDC backed by Injective USDC held by the module.
- **Pending Injective** and **Pending Noble:** Withdrawals that have been submitted but not yet settled.

In `GRADUAL` mode, total trading USDC supply must equal `noble_backing + injective_backing + pending_injective` . Pending Noble is not counted, because ICS20 burns that trading USDC when the withdrawal is sent. Pending Injective stays locked in the module until it settles. The module must also hold at least `injective_backing` in Injective USDC and at least `pending_injective` in trading USDC.

Deposits

When Injective USDC arrives over the Injective channel, the module:

1. Checks the per-transfer cap and migration ceiling.
2. Allows the IBC transfer module to credit the Injective USDC to its own account.
3. Mints the same amount of trading USDC to the receiver and records it as Injective backing.

Deposits that fail any of these checks are rejected and refunded on Injective. Noble USDC deposits are rejected while the module is active.

Withdrawals

Users withdraw trading USDC using the standard IBC transfer message, which the module intercepts:

- **To Injective:** The module locks the user's trading USDC and sends the same amount of its own Injective USDC to the requested receiver.
- **To Noble:** The transfer proceeds as before, and the module only records the withdrawal.

In both cases, the module creates a pending record and moves the amount to the pending part of the ledger. An ICS4 gate rejects any packet carrying trading USDC or module-owned Injective USDC that was not created through this path.

Settlement

When an acknowledgement or timeout for a withdrawal arrives, the module matches the packet to its pending record:

- **Success:** The locked trading USDC is burned.
- **Failure:** The user's trading USDC is unlocked, and the amount is returned to the backing.

Limits and queries

Governance configures a per-transfer cap, a ceiling on Injective backing, and a limit on the number of pending settlements. Withdrawals are also subject to the existing trading USDC rate limit. Pending settlements can be queried individually or through a paginated list.

Threat Inspection

In our threat analysis, we start by defining a set of properties required and for each property we define one or more threats. We then analyze them individually to see if they could be violated, resulting in the findings presented in the Conclusion/Findings section. Note that each threat is marked with one of the following:

- ✅ Indicates that the threat does not materialize, and no vulnerabilities have been identified in the current implementation, or
- ❌ Indicates that the threat is valid, and vulnerabilities have been identified, or
- ⚠️ Indicates that vulnerabilities exist, but they can only be exploited under the assumption arbitrary transactions are used. Scenarios restricted to whitelisted transactions do not result in exploitable conditions.

Properties

1. In active modes (`GRADUAL` and `PAUSED`), every deposit, withdrawal and settlement keeps `supply(uusdc) = NobleBacking + InjectiveBacking + PendingInjective`.

Threat 1.1: ✅ A deposit, withdrawal or settlement changes the `uusdc` supply by a different amount than `NobleBacking + InjectiveBacking + PendingInjective`.

Conclusion: The threat does not hold. Every row of the accounting table in section 8 of the guide was traced through the code. An Injective deposit credits InjectiveBacking and mints the same amount of trading USDC. An Injective withdrawal moves the amount from InjectiveBacking to PendingInjective and locks it in the module, so supply is unchanged. On success the module burns the locked amount and lowers PendingInjective; on error or timeout it unlocks the amount to the user and moves it back to InjectiveBacking, again without a supply change. On the Noble route the module only moves the amount between NobleBacking and PendingNoble, while ICS20 does the burn on send and the re-mint on refund. A refund of a Noble packet sent while DISABLED is credited to NobleBacking as a legacy refund, matching ICS20's re-mint.

The module is the only code in the chain that mints or burns trading USDC outside ICS20, and it does so in exactly two places: the deposit mint and the burn on a successful Injective withdrawal. It never mints or burns the Injective voucher; that is left to ICS20. Entering GRADUAL re-computes NobleBacking from the supply, so the equation holds by construction at the start. EndBlock re-checks it every block in GRADUAL and pauses routing if it breaks. In PAUSED the check does not run, but the only flow that writes the ledger there is settlement, which preserves the sum.

The real-app lifecycle test measures supply, all four ledger fields, the module balances and the rate limit after every step for both routes and all three outcomes, and the equation holds after each one.

Threat 1.2: ✅ An error is ignored, so the flow succeeds with only part of its ledger, bank or rate-limit changes.

Conclusion: The threat does not hold. Every call in transferInjective, transferNoble, Settle, OnAcknowledgementPacket and OnTimeoutPacket was checked. Each error is returned, and the only successful exit is after the cache context is committed. In receiveInjective every failure, including those after the ledger credit and after the ICS20 mint, ends in an error acknowledgement, and the caller commits the cache context only for a successful acknowledgement.

The inner ICS20 send and the inner callbacks, including the rate-limit middleware, run on the same cache context as the module's own writes, so a failure anywhere discards everything together. The only call whose result is not checked is the rate-limit undo, which has no return value.

Threat 1.3: ✅ A ledger write based on an old read overwrites a newer update in the same transaction.

Conclusion: The threat does not hold. Both withdrawals read the ledger before the inner transfer and write it after, so what runs in between was checked: the ICS4 gate, the rate-limit middleware and the channel keeper.

None of them touches the ledger; the gate only reads the controls. In receiveInjective the ledger is written once, before the inner receive, and nothing after that writes it again. Settle reads the ledger from the same cache context in which the inner callback has already run, and that callback does not write the ledger either.

The ledger has seven writers in total: genesis import, the two withdrawals, settlement, the legacy refund, the Injective deposit credit and the GRADUAL snapshot. None of them can run nested inside another, so no write can be based on a stale read.

Threat 1.4: ✅ `uusdc` enters or leaves dYdX by a path that changes supply but skips the ledger.

Conclusion: The threat does not hold. Trading USDC is minted in three places: by ICS20 when Noble's base denom arrives on the Noble channel, by ICS20 when a Noble withdrawal is refunded, and by the module for an Injective deposit. In active modes the first is rejected with an error acknowledgement before it reaches ICS20, the second is matched by the settlement or legacy refund credit to NobleBacking, and the third always goes through the ledger. Trading USDC returning from any other chain is an unescrow, not a mint, so supply does not change.

On the way out, trading USDC is burned only when a packet leaves on the Noble channel. In active modes every such packet passes the ICS4 gate, which admits only packets created by the decorator, and the decorator records the ledger change. Sends on any other channel are escrows that leave supply unchanged, and the decorator rejects them anyway. In PAUSED no new transfers are possible at all, while settlements of packets already in flight still update the ledger.

2. Deposits, withdrawals and settlements never leave the module holding less than it owes: physical balance ≥ `InjectiveBacking` and `uusdc` balance ≥ `PendingInjective`.

Threat 2.1: ✅ Physical vouchers leave the module without `InjectiveBacking` going down by the same amount.

Conclusion: The threat does not hold. The Injective voucher leaves the module account in exactly one place, the physical send in transferInjective, and that send happens only after InjectiveBacking has been reduced by the same amount in the same cache context. On a failed withdrawal ICS20 re-mints the voucher to the module and the module adds the amount back to InjectiveBacking, so the two move together in both directions. The module never mints or burns the voucher itself.

Nothing else in the module moves the voucher. No signed message can use the module account as sender, and the ICS4 gate rejects any packet carrying Injective USDC with the module as sender unless the decorator created it, so chain code cannot send the reserve out either. Before paying a refund, the module also checks that the physical balance still covers the whole InjectiveBacking, and in GRADUAL EndBlock checks the same every block and pauses routing if it fails.

Threat 2.2: ✅ Locked `uusdc` leaves the module other than through a burn on success or an unlock on failure.

Conclusion: The threat does not hold. The amount locked in transferInjective is written into the pending record, and settleInjective later burns or unlocks exactly that amount, read from the same record. Each record is removed in the same cache context as its settlement, and IBC core delivers at most one acknowledgement or timeout per packet, so every lock ends in exactly one burn or one unlock and never both. The unlock always goes to the sender stored in the record.

The module sends trading USDC out of its account in only two places: the unlock on failure and the deposit mint, which mints the coins and passes them straight to the receiver. Coins sent directly to the module address, whether trading USDC or the voucher, raise its balance above what the ledger records but are never paid out, since every payout is sized by the ledger and the pending record, not by the balance. In GRADUAL EndBlock checks every block that the module's trading USDC balance covers PendingInjective.

3. Pending records match `PendingInjective`, `PendingNoble` and the pending count.

Threat 3.1: ✅ `PendingInjective`, `PendingNoble` or the pending count stop matching the stored records.

Conclusion: The threat does not hold. A record is created only in the two withdrawals, where the same amount is used for the ledger change and for the record, and both are written in the same cache context and committed together. A record is removed only in Settle, which reads the amount from the record itself, subtracts it from PendingInjective or PendingNoble and deletes the record, again in one cache context.

SetPending refuses a duplicate key and increments the count only after a successful store write; RemovePending deletes the key and decrements the count in the same call, and panics on an underflow rather than wrapping. Keys are unique per route, channel and sequence, so two records cannot share a key, and IBC core delivers at most one acknowledgement or timeout per packet, so a record is settled at most once. A failed send or a failed settlement discards the cache context, so no record can be left behind without its ledger change or vice versa.

4. Only USDC originating from Injective and received through the Injective channel can be used to issue trading USDC to the packet receiver at a 1:1 ratio.

Threat 4.1: ✅ A packet containing USDC that does not originate from Injective is incorrectly treated as an Injective deposit.

Conclusion: The threat does not hold. A packet is treated as an Injective deposit only if it arrives through the bound Injective channel, contains the exact configured Injective base denom, and resolves to the configured Injective voucher. The channel is bound to Injective's connection and client, ensuring that packets received through it can only come from Injective.

The module calculates the voucher denom in the same way ICS20 does when receiving a packet. The two denom checks protect against different cases.

For example, packets that return coins to dYdX may resolve to the Injective voucher, but their packet denom does not match the configured base denom. Therefore, they are not treated as Injective deposits and are passed to ICS20, which attempts to unescrow the coins instead of minting new ones. Similarly, a packet containing only the base denom but arriving through another channel resolves to a different voucher.

Noble USDC routed through Injective, denom variations with different letter cases, other Injective assets, and Injective USDC received through other channels are all treated as separate vouchers. These packets pass through without affecting the ledger.

Threat 4.2: ✅ The issued USDC amount or the receiver does not match the amount or receiver specified in the packet.

Conclusion: The threat does not hold. The module and ICS20 use the same parser to read the packet amount. This ensures that the amount credited to the ledger, the amount of Injective USDC minted to the module, and the amount of trading USDC minted to the receiver are always equal. If the amount cannot be parsed, is zero or negative, or exceeds the per-packet limit, the packet receives an error acknowledgement.

Before rewriting the packet, the module validates the receiver as a dYdX address. Trading USDC is then minted to the account identified by that address. Addresses written in uppercase still resolve to the same account, while invalid addresses or addresses with a different prefix result in an error acknowledgement.

If sending trading USDC to the receiver fails (for example, because the receiver is a blocked module account), the deposit receives an error acknowledgement. All changes are rolled back, and Injective refunds the sender.

A deposit to the module's own address is accepted, and the minted trading USDC stays in the module. Accounting is not affected, and the existing tests show this behavior is intended.

Threat 4.3: ✅ Trading USDC is issued without the physical voucher reaching the module.

Conclusion: The threat does not hold. Before passing the packet to ICS20, the module changes only the receiver to its own address. The denom, amount, sender, and memo remain unchanged. This ensures that ICS20 mints the same amount of Injective USDC to the module.

Trading USDC is minted only after ICS20 returns a successful acknowledgement. Both operations run in the same cache, which is committed only if everything succeeds. Therefore, if receiving the packet or minting USDC fails, no physical USDC credit, ledger credit, or trading USDC is created.

Threat 4.4: ❌ `MaxTransferAmount` or `MigrationCeiling` limit can be exceeded.

Conclusion: The threat holds for the migration ceiling. Refunds of failed Injective withdrawals return the amount to the backing without checking the ceiling. Meanwhile, deposits may already have used the space freed by the pending withdrawals. Repeating this process allows the backing to exceed the ceiling without bound.

The per-transfer cap is enforced on both deposits and withdrawals. Apart from the issue described above, deposits are checked against the remaining space under the ceiling. Deposits that exceed either limit are rolled back and refunded.

Findings: Failed Injective Withdrawals Allow injective_backing to Exceed migration_ceiling Without Limit

5. Noble USDC deposits are rejected without losing funds, while other IBC transfers continue to work normally.

Threat 5.1: ✅ Noble USDC deposits are accepted and credited to users as `uusdc` instead of being rejected.

Conclusion: The threat does not hold. While routing is active (`GRADUAL` or `PAUSED`), any packet received on the Noble channel that would result in trading USDC being minted is rejected with an error acknowledgement before reaching ICS20. No tokens are minted, rate-limit capacity is not increased, and Noble refunds the sender. Therefore, no funds are lost.

The check calculates the resulting denom in the same way as ICS20, ensuring that changes or variations in the packet denom cannot bypass it.

ICS20 mints trading USDC only when Noble's base denom is received through the Noble channel. The same USDC received through another channel or routed through another chain results in a different voucher and is not credited as trading USDC.

Threat 5.2: ✅ Unrelated IBC packets are rejected or cause unintended changes to the ledger.

Conclusion: The threat does not hold. Packets that do not match either the Noble or Injective conditions are passed to the inner stack without any changes, and the ledger remains untouched.

This includes other denoms received on the Noble or Injective channels, non-canonical USDC vouchers, and trading USDC returning from other chains.

The module uses the same strict decoding rules as ICS20, so it only rejects packets that ICS20 would also reject.

6. In active modes, no packet containing trading USDC or physical USDC owned by the module can leave dYdX unless it was created by `TransferDecorator`.

Threat 6.1: ✅ A packet containing trading USDC or module-owned physical USDC passes the ICS4 gate without being created by `TransferDecorator`.

Conclusion: The threat does not hold. Every ICS20 packet passes through the gate, regardless of how the transfer was started. While the module is active, the gate requires authorization for all trading USDC transfers, regardless of the channel, and for any Injective USDC sent by the module.

ICS20 always includes the full denom trace and the sender address in lowercase in the packet. Therefore, aliases and different letter cases cannot be used to bypass the checks.

Authorization is set only by `TransferDecorator`, applies only to the single transfer it creates, and cannot be set from outside the module.

Threat 6.2: ✅ Denom or port aliases allow a packet to bypass the decorator or the ICS4 gate.

Conclusion: The threat does not hold. The decorator intercepts only the exact trading USDC bank denom. Other spellings, such as the full trace, a different letter case, or a lower-case hash, are separate bank denoms that hold no trading USDC, so a transfer using them has nothing to send. If such a send reached ICS20, the packet would still carry the trading USDC trace and the gate would reject it, because the gate checks the denom ICS20 writes into the packet and not the denom in the message.

Canonical transfers must use the transfer port, and ICS20 can only send on channels owned by that port. Both traces are built from the transfer port and the configured channels, which matches what ICS20 writes for these vouchers.

Threat 6.3: ✅ The ICS4 gate incorrectly blocks legitimate transfers.

Conclusion: The threat does not hold. The gate blocks only unauthorized packets containing the trading USDC trace or Injective USDC sent by the module. Transfers of other denoms and Injective USDC owned by users pass through unchanged.

Transfers created through the decorator are always authorized. The gate only decodes packets that were created by ICS20 itself. While the module is in `DISABLED` mode, the gate does not perform any checks. Trading USDC can only leave through the Noble and Injective routes. No trading USDC can leave while the module is in `PAUSED` mode. This is the intended behavior of the active modes.

7. An Injective withdrawal locks exactly the amount requested by the user, sends the same amount of physical USDC to the requested receiver, and records the withdrawal.

Threat 7.1: ✅ The sender, receiver, amount, or denom does not match between the user's message, the physical transfer packet, and the pending record.

Conclusion: The threat does not hold. The module locks the requested amount of trading USDC from the signer and sends the same amount of Injective USDC from its own account. It copies the receiver, memo and timeouts from the user's message unchanged. The pending record stores the user as sender, the same receiver and amount, the Injective denom, and the sequence returned by the transfer. The ledger moves the same amount from Injective backing to pending.

The amount and backing are checked before any side effect, and the lock, the send, the record and the ledger update are committed together or not at all. On settlement, the packet's amount, receiver, denom and sender are checked against the record.

The stored sender keeps the casing from the message, but on this route it is used only for the refund, which resolves to the same account.

Threat 7.2: ❌ User-controlled fields copied into the packet sent by the module can affect other users or cause incorrect refunds.

Conclusion: The threat holds. On the Injective route, the module sends the packet with itself as the sender but forwards the user's memo unchanged. As a result, contract calls triggered by the memo run under a single identity shared by all canonical withdrawers. This allows one user to access state created by another user.

On the dYdX side, refunds are paid according to the pending record and always return the locked amount to the original withdrawing user. Each settlement also modifies only its own record. Therefore, user-controlled fields cannot redirect refunds or prevent settlement.

Findings: Memo Calls on Injective Run Under One Identity Shared by All Canonical Withdrawers

Threat 7.3: ✅ The rate limit is not applied correctly to the withdrawal.

Conclusion: The threat does not hold. The physical packet contains Injective USDC, so the rate-limit middleware does not detect it as trading USDC. For this reason, the module applies the trading USDC rate limit itself, using the exact withdrawal amount before sending the packet. If the limit is exceeded, the withdrawal is rejected without any state changes.

If the withdrawal succeeds, the rate-limit charge remains in place. If it fails or times out, the module restores the charge exactly once, as part of the same operation that removes the pending record. This prevents the charge from being restored more than once.

The middleware only handles the Injective USDC denom, so it cannot restore the trading USDC rate-limit charge a second time.

8. A Noble withdrawal proceeds only while Noble withdrawals are enabled and Noble backing suffices, and moves `NobleBacking` into `PendingNoble`.

Threat 8.1: ✅ A Noble withdrawal proceeds when disabled or without enough Noble backing.

Conclusion: The threat does not hold. `transferNoble` checks the `NobleWithdrawalsEnabled` flag before it touches any state and returns `ErrDisabled` when Noble withdrawals are turned off. The subtract helper rejects any amount larger than the current `NobleBacking` with `ErrInsufficientBacking`, so backing can never go negative. `NobleBacking` moves into `PendingNoble` only inside a cache context, and that context is committed only after the inner IBC transfer succeeds. The user's own message is forwarded unchanged with the authorization flag set. The module adds no second `uusdc` rate-limit charge on this route, because only the Injective route calls `ProcessWithdrawal`.

9. The existing trading asset is preserved for users across all flows.

Threat 9.1: ✅ Users end up with the wrong USDC or with an asset they can't trade with.

Conclusion: The threat does not hold. `receiveInjective` (`ibc_middleware.go:151-162`) sends the physical Injective voucher to the module address and mints `uusdc` to the receiver only after that inner receive succeeds; on any failure the cache context is discarded. A failed Injective withdrawal (`settlement.go:90-96`) leaves the physical coins with the module and returns uusdc to the sender. In every path, the only denom the module mints, unlocks or burns is `controls.LogicalDenom`, and each flow emits events that let users track their transfers. The one exception is `MODE_DISABLED`, where packets bypass the module and an Injective deposit delivers the physical voucher, which is intended when the canonical flow is off.

Threat 9.2: ✅ The transfer response and events do not let users track their withdrawal.

Conclusion: The threat does not hold. On both routes the user gets back the inner IBC transfer's own response, so its `Sequence` is the real packet sequence on the channel the user chose; the Injective physical message changes only the token and sender. The `PendingSettlement` is stored under that channel and sequence, and on ack or timeout it is found by the packet's channel and sequence and checked against it. Each withdrawal emits events with the channel, sequence, route and `uusdc` amount, and the final outcome is reported in a settlement event keyed the same way.

10. The flows react to modes and flags exactly as intended.

Threat 10.1: ✅ `PAUSED` lets new transfers or Injective deposits through, or blocks settlement.

Conclusion: The threat does not hold. In `PAUSED`, `Transfer` (`transfer.go:32-37`) rejects new `uusdc` transfers before the authorization flag is set, and `SendPacket` (`ics4.go:25`) blocks any canonical packet that lacks the flag, so the pause cannot be bypassed. In `onRecvPacketActive`, Noble deposits get an error ack (`ibc_middleware.go:109-111`) before the mode is checked, and Injective deposits are rejected with `ErrPaused` (`115-120`); `InjectiveDepositsDisabled` guards only the Injective branch. `OnAcknowledgementPacket`, `OnTimeoutPacket` and `Settle` never check the

mode, so in-flight packets still settle and refunds still go through. `DISABLED` passes everything straight through.

Threat 10.2: ✅ `DISABLED` mode is not a full pass-through.

Conclusion: The threat does not hold. In `DISABLED`, `OnRecvPacket` (`ibc_middleware.go:87-89`) hands the original packet to the inner app with no cache context, `Transfer` (`transfer.go:32-34`) forwards the user's message unchanged, and `SendPacket` (`ics4.go:25`) skips the authorization gate entirely. When no pending record exists, `OnAcknowledgementPacket` and `OnTimeoutPacket` go through `forwardUnsettled`. There, `legacyNobleRefund` returns false for `DISABLED`, so the callback is forwarded with the original context and no ledger credit. The only side effects are the extra reads of the controls and the pending store, which cost some gas but change no state or behavior.

11. Limits cannot be abused to deny service to other users.

Threat 11.1: ❌ Pending slots can be exhausted cheaply.

Conclusion: The threat holds. Anyone can fill all `MaxPendingSettlements` slots for about one tx fee per slot, since there is no minimum amount. Noble runs packet-forward-middleware (mainnet v11.6.0, PFM v8.2.0), and the module forwards the user's memo unchanged. So a dust withdrawal with a forward memo and a long timeout keeps its slot taken for as long as the attacker chooses, at no further cost. It blocks new canonical withdrawals for all users on both routes, but deposits, settlement and refunds are not affected and no funds are at risk. Possible mitigations are rejecting forward memos on canonical routes, a minimum withdrawal amount, or a per-sender pending limit.

Findings: *Pending Cap Exhaustion*

Threat 11.2: ❌ Route backing can be tied up by one user.

Conclusion: The threat holds, but needs capital roughly equal to the route's backing. Timeouts are user-chosen and the user's memo is forwarded unchanged. Noble runs packet-forward-middleware and Injective's binary includes it, so a withdrawal with a forward memo can stay in `PendingNoble` or `PendingInjective` for as long as the user chooses. Other users are blocked only once the attacker has drained that route's backing with their own uusdc. `MaxTransferAmount` and x/ratelimit slow this down but do not prevent it. The funds aren't lost, only locked, so a well-funded holder could do it. The mitigation is rejecting forward memos on canonical routes.

Findings: *Withdraw to Injective and Redeposit Loop Drains the Shared uusdc Rate-Limit Capacity*

12. The module classifies acks and timeouts exactly as ICS20 does.

Threat 12.1: ✅ The module and ICS20 disagree on whether a packet was refunded.

Conclusion: The threat does not hold. `refundsSender` treats only `Acknowledgement_Error` as a refund, which matches ibc-go v8 ICS20 (`relay.go:316-325`). Result, empty-result and missing-response acks count as delivered in both, and undecodable or non-canonical bytes make the callback fail before `Settle` runs, so nothing is written. In `OnAcknowledgementPacket` (`ibc_middleware.go:191-206`), the module reads the outcome from the same ack bytes it passes unchanged to the inner app, in the same callback and cache context, and settles only after the inner app succeeds. Timeouts always settle as refunds, as in ICS20.

13. Each settlement outcome updates the correct state for its route

Threat 13.1: ✅ A settlement pays the wrong recipient or amount, or updates the wrong ledger field.

Conclusion:

On successful Injective withdrawals, the module burns the trading USDC, while ICS20 has already burned the Injective USDC.

On failure (error acknowledgement or timeout), ICS20 refunds the Injective USDC to the module. The module restores the Injective backing and returns the trading USDC to the original user.

On the Noble route, the module clears the pending amount. If the transfer fails, Noble backing is restored because ICS20 refunds the coins directly to the user.

The supply equation holds after every outcome. Double refunds are prevented by removing the pending record during settlement, while IBC core prevents duplicate acknowledgements or timeouts.

Both the module and ICS20 treat only error acknowledgements as failures.

Threat 13.2: ✅ The rate limit is not restored on failure, or is restored more than once.

Conclusion:

For Injective withdrawals, the module charges the USDC rate limit and restores it once on failure. The middleware only sees the Injective denom, so it cannot refund the USDC charge again.

For Noble withdrawals, the middleware handles both the charge and refund; the module does not touch the rate limit.

14. Every canonical packet can be settled, regardless of relay order

Threat 14.1: ❌ Module checks can permanently prevent an acknowledgement or timeout from being processed.

Conclusion:

A Noble withdrawal from an uppercase address never settles. The module stores the sender as entered, while ICS20 puts its lowercase form in the packet. The sender comparison therefore fails on every acknowledgement and timeout.

The withdrawal stays pending, its refund is blocked, and its slot is never released. An attacker can fill all slots with small withdrawals and block withdrawals on both routes.

Failed Injective withdrawals also depend on the module's entire Injective USDC balance being at least equal to the full Injective backing, so a shortfall would block all failed Injective settlements. This is not a concern because such a shortfall can only occur if the system is already in an invalid state.

Other error and panic paths are unreachable in valid states or are standard ICS20 behavior. Settlement callbacks also ignore the module's operating mode, so pausing the module or disabling Noble withdrawals does not block settlement.

Findings: *Noble Withdrawal from an Uppercase Address Never Settles*

Threat 14.2: ✅ Different orders of acknowledgements, timeouts, and new transfers cause later settlements to fail or produce different final states.

Conclusion:

Settlement order does not affect success or the final state of withdrawals already in flight. Each settlement changes the ledger only by its own amount, and the existing random-order property test confirms this across deposits, withdrawals, and mode changes.

New deposits and withdrawals can still have different outcomes depending on when refunds arrive and how much backing and pending capacity are available. For example, a refund may push the backing above the ceiling, temporarily blocking new deposits. This is expected behavior, as refunds must never be blocked.

15. Packets are matched to the correct pending records

Threat 15.1: ✅ A packet is matched to the wrong pending record, or a canonical packet is processed without its record.

Conclusion:

Records are uniquely identified by channel and sequence. The two routes use different channels, and sequences are unique within each channel.

The pending record is saved using the sequence returned by the transfer, in the same transaction as the send. Thus, every successfully sent packet has a record.

While the module is active, all USDC transfers on canonical channels go through it. Packets sent while it is disabled are the only exception, by design.

The stored sender can still differ from the packet sender due to address casing. This is covered by Threat 14.1.

16. Settlement queries return correct and complete results

Threat 16.1: ✅ Queries return incorrect data, behave unexpectedly for unusual inputs, or are too expensive.

Conclusion:

Empty requests return invalid-argument, and unknown channels or sequences return not-found. Pagination and reverse ordering follow standard SDK behavior.

Queries without a limit count all records, but the pending limit keeps the cost bounded and does not affect consensus.

17. Helper functions used by the flows behave correctly

Threat 17.1: ✅ Amount conversion, denom trace construction, storage key encoding, default values, or record validation is incorrect.

Conclusion:

Amount conversions reject negative and oversized values and preserve amounts without loss. Denom traces match ICS20 expectations, enforced by configuration checks.

Storage keys are collision-free and sort by route, channel, and sequence. Defaults leave the module disabled with an empty ledger and no pending records.

Record validation relies on senders, receivers, and denoms coming from validated input. However, it stores the sender's original casing, causing the issue described in Threat 14.1.

18. Events accurately describe what happened

Threat 18.1: ❌ Events are missing or contain misleading information.

Conclusion:

All four documented events are emitted only when the action is committed: `canonical_usdc_transfer`, `ibc_transfer`, `canonical_usdc_inbound`, and `canonical_usdc_settlement`.

Some details are missing for indexers: transfer events contain no amount, sender, or receiver. Injective withdrawals emit two `ibc_transfer` events, one naming the module, while deposit events identify the module as the receiver.

Findings: *Events Are Incomplete for Indexers*

Findings

Title	Type	Severity	Status	Issue
Noble Withdrawal from an Uppercase Address Never Settles	IMPLEMENTATION	4 CRITICAL	RESOLVED	https://github.com/tac0turtle/v4-chain/pull/7
Pending Cap Exhaustion	IMPLEMENTATION	3 HIGH	RESOLVED	https://github.com/tac0turtle/v4-chain/pull/9
Withdraw to Injective and Redeposit Loop Drains the Shared uusdc Rate-Limit Capacity	IMPLEMENTATION	2 MEDIUM	RESOLVED	https://github.com/tac0turtle/v4-chain/pull/10
Memo Calls on Injective Run Under One Identity Shared by All Canonical Withdrawers	PROTOCOL	2 MEDIUM	RESOLVED	https://github.com/tac0turtle/v4-chain/pull/9
Failed Injective Withdrawals Allow injective_backing to Exceed migration_ceiling Without Limit	IMPLEMENTATION	1 LOW	RESOLVED	https://github.com/tac0turtle/v4-chain/pull/8
Events Are Incomplete for Indexers	IMPLEMENTATION	0 INFORMATIONAL	RESOLVED	https://github.com/tac0turtle/v4-chain/pull/9
Code Improvements	PRACTICE	0 INFORMATIONAL	RESOLVED	https://github.com/tac0turtle/v4-chain/pull/9

Noble Withdrawal from an Uppercase Address Never Settles

Project	dYdX 2026 Q4: Canonical USDC
Type	IMPLEMENTATION
Severity	4 CRITICAL
Impact	3 HIGH
Exploitability	3 HIGH
Status	RESOLVED
Issue(s)	https://github.com/tac0turtle/v4-chain/pull/7

Involved Artifacts:

- x/canonicalusdc/keeper/
- x/canonicalusdc/ibc_middleware.go

Description:

Bech32 accepts fully uppercase addresses, so `MsgTransfer.ValidateBasic` allows users to start Noble withdrawals with uppercase sender addresses.

ICS20 converts the sender to lowercase when writing it to the packet (`sender.String()`), but `newPending` stores it exactly as provided by the user. When an acknowledgement or timeout arrives, `validatePendingPacket` detects the casing mismatch and returns an error before ICS20 can process the packet. Because of this, IBC core rejects every `MsgAcknowledgement`, `MsgTimeout`, and `MsgTimeoutOnClose` for that packet, so relayer cannot settle it. If the transfer failed, the user's refund is never minted; `PendingNoble` remains above zero and the record continues to occupy one of the shared `MaxPendingSettlements` slots. An attacker can fill all slots with 1-unit withdrawals at the cost of transaction fees, blocking withdrawals on both the Injective and Noble routes and preventing governance from disabling Noble withdrawals or changing the route. At first glance, this looks like an impossible internal error, but a user can trigger it with a perfectly valid transaction, for example, by copying an uppercase address from a QR code. Impact is **HIGH** because funds and withdrawals remain frozen until a chain upgrade, and exploitability is **HIGH** because any user can trigger it.

Problem Scenarios:

A PoC in the form of a Go test file can be provided to reproduce all the effects described below.

1. **Refunds can get stuck.** A user initiates a Noble withdrawal using an uppercase address. If the packet times out or receives an error acknowledgement, the refund is never processed because ICS20 only handles refunds after `validatePendingPacket` succeeds. As a result, the user's USDC remains stuck, and the corresponding `PendingNoble` entry is never cleared.
2. **Withdrawals can become unavailable on both routes.** An attacker can fill all `MaxPendingSettlements` slots with small Noble withdrawals that cannot be settled. Once the limit is reached, all new withdrawals fail with `ErrPendingLimit`, regardless of whether they use

the Injective or Noble route. This effectively blocks withdrawals for everyone. The cost of carrying out the attack is relatively low: transaction fees and 1 unit of USDC per pending withdrawal.

3. **Governance can get stuck.** If `PendingNoble` never reaches zero, governance cannot disable Noble withdrawals because the operation fails with `ErrNobleBackingRemains`. It also cannot change the withdrawal route or disable the module while pending settlements remain. This can leave the protocol stuck in its current configuration until the pending state is resolved.

Recommendation:

Store the sender in canonical form in `newPending`. `msg.ValidateBasic()` already ran at `transfer.go:38`, so the parse cannot fail:

```
Sender: sdk.MustAccAddressFromBech32(msg.Sender).String(),
```

Alternatively, compare the parsed addresses in `validatePendingPacket` instead of the strings.

Pending Cap Exhaustion

Project	dYdX 2026 Q4: Canonical USDC
Type	IMPLEMENTATION
Severity	3 HIGH
Impact	3 HIGH
Exploitability	2 MEDIUM
Status	RESOLVED
Issue(s)	https://github.com/tac0turtle/v4-chain/pull/9

Involved Artifacts:

- x/canonicalusdc/keeper/

Description:

Canonical withdrawals forward the user's memo and timeout to ICS20 unchanged. Noble runs packet-forward-middleware (PFM v8.2.0 on mainnet), and so does Injective's binary. When the memo carries a forward instruction, the destination holds the ack until the next hop settles, and the user sets that hop's timeout. Once a packet has been received, it can no longer time out on dYdX, so its pending settlement stays open.

`max_pending_settlements` is one global cap for both routes, with no minimum amount. An attacker can fill every slot with 1 `uusdc` withdrawals carrying forward memos, at one tx fee per slot, and keep them full for as long as they choose. Relaying doesn't free them. Governance can't recover: with settlements outstanding, the module can't be `DISABLED` or its routes changed, and a raised cap is refilled at one fee per slot. In `GRADUAL`, `uusdc` can leave only over the Noble and Injective channels, so no user can withdraw USDC until a software upgrade. With enough capital, the same delay can also lock a route's backing in pending.

Impact is HIGH: all USDC withdrawals are blocked for as long as the attacker chooses, and recovery needs an upgrade, though no funds are lost. Exploitability is HIGH: anyone can do it for a few fees, and Noble's PFM is live on mainnet.

Problem Scenarios:

An attacker sends `max_pending_settlements` Noble withdrawals of 1 `uusdc`, each with a memo like `{"forward":{"receiver":"…","port":"transfer","channel":"channel-X","timeout":"8760h"}}`. Noble holds every ack, all slots stay full, and every new canonical withdrawal on both routes fails with `ErrPendingLimit` for up to a year.

Governance raises `max_pending_settlements` to restore service. The attacker fills the new slots the same way, at one fee per slot. A proposal to switch the module to `DISABLED` is rejected because settlements are outstanding.

A well-funded holder withdraws most of `NobleBacking` or `InjectiveBacking` with forward memos and long forward timeouts. Backing stays in pending, and other users' withdrawals on that route fail with `ErrInsufficientBacking`.

What happens on the dYdX side: forward memos are accepted and forwarded unchanged, unacked slots are never freed, and other users are blocked on both routes. The ack hold on Noble follows from its PFM wiring and was not run end to end.

Recommendation:

In `TransferDecorator.Transfer`, reject canonical transfers whose memo has a top-level forward key, decoding the memo into a map the way PFM does, so the key match is exact. Add a minimum withdrawal amount or a per-sender pending limit so dust can't fill the shared cap. A timeout cap does not help, because a received packet can no longer time out. Add tests for both routes: a forward memo is rejected, and a plain-text memo or a JSON memo without forward is accepted. Ship the fix before the module enters `GRADUAL` on mainnet; if it is exploited earlier, recovery is an upgrade with the fix followed by a raised cap.

Withdraw to Injective and Redeposit Loop Drains the Shared uusdc Rate-Limit Capacity

Project	dYdX 2026 Q4: Canonical USDC
Type	IMPLEMENTATION
Severity	2 MEDIUM
Impact	2 MEDIUM
Exploitability	2 MEDIUM
Status	RESOLVED
Issue(s)	https://github.com/tac0turtle/v4-chain/pull/10

Involved Artifacts:

- x/canonicalusdc/
- x/ratelimit/

Description:

The x/ratelimit module keeps one withdrawal bucket per denom. Before this module, a withdrawal of `uusdc` consumed capacity from the `uusdc` bucket, and a deposit of `uusdc` gave it back, so sending USDC out and bringing it back in was neutral for the rate limit.

With x/canonicalusdc in `GRADUAL` mode this is no longer the case. Every canonical withdrawal, to Noble or to Injective, is still charged to the `uusdc` bucket. The Injective route does this explicitly, because the physical packet carries the Injective denom and x/ratelimit would otherwise not charge it, protocol/x/canonicalusdc/keeper/transfer.go#L96-L100:

```
// The physical packet carries the Injective denom, so x/ratelimit does not
// charge it against logical USDC capacity.
if err := d.keeper.ratelimitKeeper.ProcessWithdrawal(cacheCtx, controls.LogicalDenom,
amount.BigInt()); err != nil {
    return nil, err
}
```

Deposits, however, no longer refill that bucket. An Injective deposit arrives as the Injective voucher, and x/ratelimit credits the capacity of the denom it sees in the packet, which is the physical denom and not `uusdc`, protocol/x/ratelimit/ibc_middleware.go#L141:

```
im.keeper.IncrementCapacitiesForDenom(ctx, ibcTransferPacketInfo.Denom,
ibcTransferPacketInfo.Amount)
```

Noble deposits are rejected while in `GRADUAL` (protocol/x/canonicalusdc/ibc_middleware.go#L109-L111), so they cannot refill it either. The only thing that restores `uusdc` capacity is time.

K6 in the audit guide documents this asymmetry and explains why it is wanted: charging only outflows means a counterfeit Injective deposit cannot create withdrawal capacity. The guide, however, only discusses the effect on the migration pace. It does not mention that the same mechanism lets a single user with enough USDC to fill the hourly bucket consume the whole bucket, and that this is a regression compared with the current chain, where the same round trip is harmless.

Problem Scenarios:

Mallory holds C USDC on dYdX and wants to block withdrawals for everyone.

1. Mallory withdraws C `uusdc` to Injective. The `uusdc` bucket drops by C. The packet is relayed and acknowledged, so the pending record settles and the locked `uusdc` is burned.
2. Mallory sends the same C back from Injective. The deposit mints C `uusdc` for her and the ledger returns to where it was. The `uusdc` bucket does not move, because x/ratelimit credited the Injective denom instead.
3. Mallory repeats steps 1 and 2. Each lap costs her transaction fees and relay time, and the capital comes back every time.

With the mainnet limits of 1M per hour and 10M per day, a few laps with 1M are enough to keep the hourly bucket at zero, and a day of looping drains the daily one. From that point every withdrawal by any other user, on both routes, fails with `ErrWithdrawalExceedsCapacity` until the bucket refills. Nothing looks wrong on-chain: the ledger balances, so the auto-pause never fires. The only symptom is that withdrawals fail.

Recommendation:

Let a canonical Injective deposit refill the `uusdc` bucket, but only up to what the Injective route recently consumed, for example the route's net outflow in the current window. That keeps the property K6 is after, since a counterfeit deposit still cannot create capacity that was never spent, while an honest round trip stops costing everyone else.

Memo Calls on Injective Run Under One Identity Shared by All Canonical Withdrawers

Project	dYdX 2026 Q4: Canonical USDC
Type	PROTOCOL
Severity	2 MEDIUM
Impact	3 HIGH
Exploitability	1 LOW
Status	RESOLVED
Issue(s)	https://github.com/tac0turtle/v4-chain/pull/9

Involved Artifacts:

- x/canonicalusdc/keeper/transfer.go

Description:

On the Injective route the decorator does not send the user's `MsgTransfer`. It takes a copy, swaps the token for the Injective voucher and the sender for the module account, and sends that, protocol/x/canonicalusdc/keeper/transfer.go#L105-L107:

```
physical := *msg
physical.Token = sdk.NewCoin(controls.InjectiveDenom, amount)
physical.Sender = types.ModuleAddress.String()
```

Everything else in the message, including the memo and the receiver, goes into the packet as the user wrote it, and the module puts no restriction on the memo.

The problem shows up on the Injective side through IBC hooks. A memo of the form `{"wasm": {"contract": ..., "msg": ...}}` makes Injective call that contract after the transfer lands. The contract does not see the dYdX user. It sees an intermediate address derived from the destination channel and the packet sender (`DeriveIntermediateSender`), and that derivation exists precisely so that each remote sender gets its own local identity.

On this route the packet sender is always the module account. So every dYdX user who withdraws to Injective with a hook memo acts on Injective as the same address, call it D. Whatever one user builds at D through a hook, a lending deposit, a cw20 balance, an allowance, a position in a vault, belongs to D, and any other user can reach it by sending a canonical withdrawal with a memo that calls the same contract. The Noble route and the legacy `DISABLED` path send the user's own message, so there each user keeps a distinct identity. Nothing on the dYdX side goes wrong: both packets ack with success and the locked `uusdc` is burned as for any delivery. The loss happens on Injective.

For anyone to get hurt, a hook memo has to end up on a canonical withdrawal first, either written by the user or, more likely, built by a frontend or partner flow that does not know the sender is the module. That is why the exploitability is low: the attack itself is trivial, but it only works after someone has made that mistake.

Problem Scenarios:

The scenario below is one example. Any Injective contract that keeps state per caller, a vault, a staking contract, a cw20 allowance, gives a similar one. Assume `GRADUAL` with the Injective route bound and IBC hooks active on Injective.

1. Alice withdraws 1,000 USDC on the Injective channel, with an Injective lending contract C as receiver and memo `{"wasm":{"contract":"C","msg":{"deposit":{}}}}`. On Injective, C receives the 1,000 USDC and credits the deposit to `info.sender`, which is D.
2. The ack succeeds. dYdX burns Alice's locked 1,000 `uusdc` as usual.
3. Mallory withdraws 1 base unit to the same contract with memo `{"wasm":{"contract":"C","msg":{"withdraw":{"amount":"1000000000","recipient":"inj1mallory"}}}}`. The call runs as D, so C pays Alice's 1,000 USDC to Mallory.
4. When Alice later tries to withdraw through her own hook, there is nothing left at D.

A contract that pays out only to `info.sender` does not lose the funds to Mallory, but it freezes them at D, which no key controls.

Recommendation:

Do not let a user's memo run under the module's identity. The simplest option is to reject Injective-route transfers with a non-empty memo. Rejecting is better than silently clearing the memo, because a cleared memo makes a partner flow fail quietly on the Injective side, while a rejected transaction tells the user and the frontend right away.

If memos are needed on this route, allow only shapes that Injective does not act on, and document that Injective-route packets carry the module account as sender.

Failed Injective Withdrawals Allow injective_backing to Exceed migration_ceiling Without Limit

Project	dYdX 2026 Q4: Canonical USDC
Type	IMPLEMENTATION
Severity	1 LOW
Impact	1 LOW
Exploitability	2 MEDIUM
Status	RESOLVED
Issue(s)	https://github.com/tac0turtle/v4-chain/pull/8

Involved Artifacts:

- x/canonicalusdc/keeper/

Description:

`migration_ceiling` is documented as the upper limit for `injective_backing`, which represents how much trading USDC can be backed by Injective-origin USDC. `CreditInjectiveInbound` enforces this limit when processing deposits, but compares the ceiling with `injective_backing` alone.

When a user withdraws Injective USDC, the amount is moved from `injective_backing` to `pending_injective` until the withdrawal is settled. This frees up space under the ceiling, allowing new deposits to be accepted.

If the withdrawal fails, `settleInjective` returns the amount to `injective_backing` without checking the ceiling. Skipping this check is reasonable because refunds must not be blocked. However, the refunded amount is added on top of the new deposits that already filled the freed space.

The amount by which the ceiling can be exceeded is not limited by the amount pending at any given time. During each round of withdrawals, deposits, and failed withdrawals, the backing can increase by the amount of the pending withdrawals. This amount is limited only by the pending settlement limit, the maximum transfer amount, and the trading USDC rate limit.

Problem Scenarios:

Assume the ceiling `C` is fully used (`injective_backing = C`). A user withdraws the entire amount to Injective with a short timeout, causing the packets to time out. During this time, `injective_backing` drops to 0.

While the withdrawals are pending, the user deposits another `C` of Injective USDC. This deposit passes the ceiling check because the current backing is 0.

When the withdrawals time out, the refunds restore the original backing, resulting in `injective_backing = 2C`, with nothing pending. Each additional round can increase the backing by another `C`.

Apart from transaction fees, the user loses nothing.

The issue was confirmed with a test that runs these rounds through the full application, from the transfer message to acknowledgement handling. Starting from a filled ceiling of 3M, `injective_backing` reached 6M, 9M and 12M after three rounds, with no pending settlements and the backing invariant still holding.

As a result, the ceiling no longer limits the protocol's exposure to Injective-origin USDC. While the backing remains above the ceiling, all new Injective deposits from other users are also rejected.

Recommendation:

In `CreditInjectiveInbound`, compare the ceiling against `injective_backing + pending_injective`. Withdrawals and refunds do not change that sum, so the ceiling becomes a hard limit, while refunds still do not require a ceiling check and cannot be blocked.

Add a test that repeats the withdraw, deposit and failed-withdrawal cycle and asserts the sum stays within the ceiling.

Events Are Incomplete for Indexers

Project	dYdX 2026 Q4: Canonical USDC
Type	IMPLEMENTATION
Severity	0 INFORMATIONAL
Impact	0 NONE
Exploitability	3 HIGH
Status	RESOLVED
Issue(s)	https://github.com/tac0turtle/v4-chain/pull/9

Involved Artifacts:

- x/canonicalusdc/keeper/
- x/canonicalusdc/ibc_middleware.go

Description:

The module emits all four documented events, but some lack information needed by indexers. `canonical_usdc_transfer` does not include the amount, sender, or receiver, while `canonical_usdc_settlement` omits the receiver and denoms. Indexers must join these events with others using the transaction, channel, and sequence.

Injective withdrawals emit two `ibc_transfer` events: one identifies the user as the sender, the other identifies the module account. This can cause indexers to count the withdrawal twice or attribute it to the module.

Injective deposit events also identify the module as the receiver. Only `canonical_usdc_inbound` identifies the actual user.

Impact is NONE: all information is available on-chain; only off-chain indexing is affected.

Exploitability is HIGH: these event limitations occur on every canonical transfer.

Problem Scenarios:

1. An indexer reads `canonical_usdc_transfer` to track withdrawals. It cannot show the amount or the user without parsing other events.
2. An indexer that counts `ibc_transfer` events records an Injective withdrawal twice, once from the user and once from the module.
3. An indexer that reads the ICS20 receive events credits every Injective deposit to the module account instead of the user.

Recommendation:

Add `amount`, `sender` and `receiver` to `canonical_usdc_transfer`, and `receiver` and the denoms to `canonical_usdc_settlement`. Document which events indexers should use for canonical transfers, and that the ICS20 events on the Injective route name the module account.

Code Improvements

Project	dYdX 2026 Q4: Canonical USDC
Type	PRACTICE
Severity	0 INFORMATIONAL
Impact	0 NONE
Exploitability	0 NONE
Status	RESOLVED
Issue(s)	https://github.com/tac0turtle/v4-chain/pull/9

Involved Artifacts:

- app/
- app/upgrades/
- x/canonicalusdc/
- x/canonicalusdc/keeper/
- x/canonicalusdc/keeper/types/

Description:

- Both the Noble and Injective routes check the pending settlement limit only after submitting the packet. As a result, gas is wasted on transactions that should have been rejected.
- ErrDisabled is returned when NobleWithdrawalsEnabled is false, but its message says that canonical USDC routing is disabled. This does not accurately describe the condition that triggered the error.
- This code maps every error returned by `GetPendingPage` to `codes.InvalidArgument`. However, GetPendingPage can return two types of errors:
  - an invalid pagination key, which is a user error that should be returned as `codes.InvalidArgument`;
  - a failure to decode a stored record using `k.cdc.Unmarshal`, which is an internal error and should return `codes.Internal`.
- Add tests that make a canonical withdrawal through authz MsgExec and through the ICA host. Each should check the pending record's sender, the ledger update, and settlement on success, error ack and timeout.
- The module does not pass the repository's configured `golangci-lint` checks. We recommend running `make lint` on the module and fixing the reported issues before merging.
- Several module errors are wrapped with `fmt.Errorf("%w: ...")` instead of `errorsmod.Wrap`. The value `fmt.Errorf` returns has no `Cause()`, so `ABCIInfo` stops at it and the transaction result reports codespace `undefined`, code 1, instead of the module's codespace and code. The log still carries the full text. The rest of the repository uses `errorsmod.Wrap` for this. In addition, `validatePendingPacket` returns plain `fmt.Errorf` errors with no registered module error at all, so a relayer's ack or timeout transaction that fails there has no module code either.

Appendix: Vulnerability Classification

For classifying vulnerabilities identified in the findings of this report, we employ the simplified version of Common Vulnerability Scoring System (CVSS) v3.1, which is an industry standard vulnerability metric. For each identified vulnerability we assess the scores from the *Base Metric Group*, the Impact score, and the Exploitability score. The *Exploitability score* reflects the ease and technical means by which the vulnerability can be exploited. That is, it represents characteristics of the *thing that is vulnerable*, which we refer to formally as the *vulnerable component*. The *Impact score* reflects the direct consequence of a successful exploit, and represents the consequence to the *thing that suffers the impact*, which we refer to formally as the *impacted component*. In order to ease score understanding, we employ CVSS Qualitative Severity Rating Scale, and abstract numerical scores into the textual representation; we construct the final *Severity score* based on the combination of the Impact and Exploitability sub-scores.

As blockchains are a fast evolving field, we evaluate the scores not only for the present state of the system, but also for the state that deems achievable within 1 year of projected system evolution. E.g., if at present the system interacts with 1-2 other blockchains, but plans to expand interaction to 10-20 within the next year, we evaluate the impact, exploitability, and severity scores wrt. the latter state, in order to give the system designers better understanding of the vulnerabilities that need to be addressed in the near future.

Impact Score

The Impact score captures the effects of a successfully exploited vulnerability on the component that suffers the worst outcome that is most directly and predictably associated with the attack.

Impact Score	Examples
🟠 **High**	Halting of the chain; loss, locking, or unauthorized withdrawal of funds of many users; arbitrary transaction execution; forging of user messages / circumvention of authorization logic
🟡 **Medium**	Temporary denial of service / substantial unexpected delays in processing user requests (e.g. many hours/days); loss, locking, or unauthorized withdrawal of funds of a single user / few users; failures during transaction execution (e.g. out of gas errors); substantial increase in node computational requirements (e.g. 10x)
🟢 **Low**	Transient unexpected delays in processing user requests (e.g. minutes/a few hours); Medium increase in node computational requirements (e.g. 2x); any kind of problem that affects end users, but can be repaired by manual intervention (e.g. a special transaction)
🔵 **None**	Small increase in node computational requirements (e.g. 20%); code inefficiencies; bad code practices; lack/incompleteness of tests; lack/incompleteness of documentation

Exploitability Score

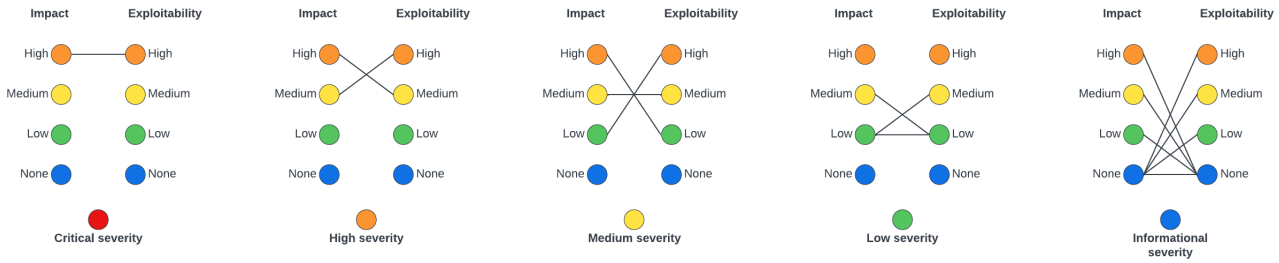
The Exploitability score reflects the ease and technical means by which the vulnerability can be exploited; it represents the characteristics of the vulnerable component. In the below table we list, for each category, examples of actions by actors that are enough to trigger the exploit. In the examples below:

- *Actors* can be any entity that interacts with the system: other blockchains, system users, validators, relayers, but also uncontrollable phenomena (e.g. network delays or partitions).
- *Actions* can be
    - *legitimate*, e.g. submission of a transaction that follows protocol rules by a user; delegation/redelegation/bonding/unbonding; validator downtime; validator voting on a single, but alternative block; delays in relaying certain messages, or speeding up relaying other messages;
    - *illegitimate*, e.g. submission of a specially crafted transaction (not following the protocol, or e.g. with large/incorrect values); voting on two different alternative blocks; alteration of relayed messages.
- We employ also a *qualitative measure* representing the amount of certain class of power (e.g. possessed tokens, validator power, relayed messages): *small* for < 3%; *medium* for 3-10%; *large* for 10-33%, *all* for >33%. We further quantify this qualitative measure as relative to the largest of the system components. (e.g. when two blockchains are interacting, one with a large capitalization, and another with a small capitalization, we employ *small* wrt. the number of tokens held, if it is small wrt. the large blockchain, even if it is large wrt. the small blockchain)

Exploitability Score	Examples
🟠 **High**	illegitimate actions taken by a small group of actors; possibly coordinated with legitimate actions taken by a medium group of actors
🟡 **Medium**	illegitimate actions taken by a medium group of actors; possibly coordinated with legitimate actions taken by a large group of actors
🟢 **Low**	illegitimate actions taken by a large group of actors; possibly coordinated with legitimate actions taken by all actors
🔵 **None**	illegitimate actions taken in a coordinated fashion by all actors

Severity Score

The severity score combines the above two sub-scores into a single value, and roughly represents the probability of the system suffering a severe impact with time; thus it also represents the measure of the urgency or order in which vulnerabilities need to be addressed. We assess the severity according to the combination scheme represented graphically below.



As can be seen from the image above, only a combination of high impact with high exploitability results in a Critical severity score; such vulnerabilities need to be addressed ASAP. Accordingly, High severity score receive vulnerabilities with the combination of high impact and medium exploitability, or medium impact, but high exploitability.

Severity Score	Examples
🔴 **Critical**	Halting of chain via a submission of a specially crafted transaction
🟠 **High**	Permanent loss of user funds via a combination of submitting a specially crafted transaction with delaying of certain messages by a large portion of relayers
🟡 **Medium**	Substantial unexpected delays in processing user requests via a combination of delaying of certain messages by a large group of relayers with coordinated withdrawal of funds by a large group of users
🟢 **Low**	2x increase in node computational requirements via coordinated withdrawal of all user tokens
🔵**Informational**	Code inefficiencies; bad code practices; lack/incompleteness of tests; lack/incompleteness of documentation; any exploit for which a coordinated illegitimate action of all actors is necessary

Disclaimer

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability, etc.) set forth in the associated Services Agreement. This report provided in connection with the Services set forth in the Services Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement.

This audit report is provided on an “as is” basis, with no guarantee of the completeness, accuracy, timeliness or of the results obtained by use of the information provided. Informal has relied upon information and data provided by the client, and is not responsible for any errors or omissions in such information and data or results obtained from the use of that information or conclusions in this report. Informal makes no warranty of any kind, express or implied, regarding the accuracy, adequacy, validity, reliability, availability or completeness of this report. This report should not be considered or utilized as a complete assessment of the overall utility, security or bugfree status of the code.

This audit report contains confidential information and is only intended for use by the client. Reuse or republication of the audit report other than as authorized by the client is prohibited.

This report is not, nor should it be considered, an “endorsement”, “approval” or “disapproval” of any particular project or team. This report is not, nor should it be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts with Informal to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor does it provide any indication of the client’s business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should it be leveraged as investment advice of any sort.

Blockchain technology and cryptographic assets in general and by definition present a high level of ongoing risk. Client is responsible for its own due diligence and continuing security in this regard.